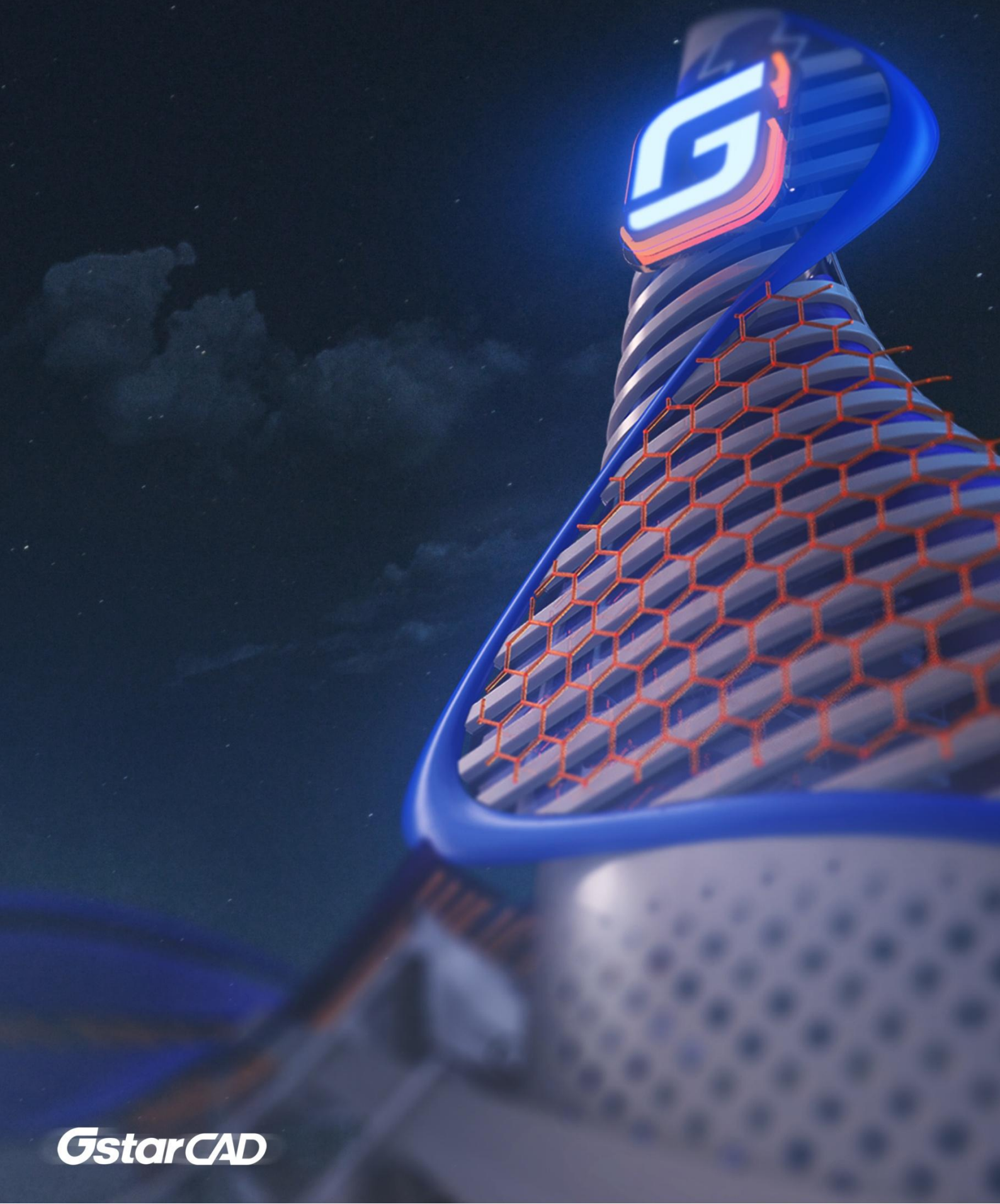
GRX Development Guide

GstarCAD 2020



GstarCAD

Table of Contents

1. What is GRX? 4
2. Development Environment 5
3. Install GRX SDK....6
4. Development Instruction....7
   4.1. Create a GRX Project under win 64-Bit 7
      4.1.1. Run Microsoft Visual Studio 2017....7
      4.1.2. Input Project Saved Path and Project Name 7
      4.1.3. Create a New cpp File 7
      4.1.4. Set the Compile and Link Configuration 9
      4.1.5. Add Codes....10
      4.1.6. Compile Programming....12
      4.1.7. Run Program 12
   4.2. GRX Program with MFC Library 13
      4.2.1. Run Microsoft Visual Studio 2017....13
      4.2.2. Input Project Saved Path and Project Name 13
      4.2.3. Choose DLL Type 13
      4.2.4. Finish the Project Creation 14
      4.2.5. Create a New Resource and Corresponding Class 14
      4.2.6. Create Three New C++ Class Files 16
      4.2.7. Add Codes....17
      4.2.8. Set Compile and Link Item....20
      4.2.9. Compile Programming....22
      4.2.10. Run Program....22
5. What is Project Property Sheet File....24
6. How to Use Project Property Sheet File 25
7. GstarCAD Programming General Method to Adapt 4K Monitor 27
   7.1. Development Background and Target....27

7.2. Implementation Method 27
7.2.1. Dialog Box, User-Defined Window, Control Adaptive 27
7.2.2. Image Adaptation 27
8. The differences between GRX and ARX interface 28
8.1. Function: monitorInputPoint 28
8.2. Function: intersectWith 30
8.3. Function: AcDbField::evaluate 30
8.4. Interface for “Plot” 32
8.5. Gs initialization 33
8.6. Gs Pan 35
8.7. acedGetArgs() release issue 35
9. Comparison of GRX and ARX libraries 36
9.1. GRX class library description 36
9.1.1. GcRx 36
9.1.2. GcEd 36
9.1.3. GcDb 37
9.1.4. GcGi 37
9.1.5. GcGe 37
10. Copyright 38

1. What is GRX?

GRX is the RX (Runtime eXtension) programming environment of GstarCAD®, which is the latest secondary development package released by GstarCAD®. It provides Object-oriented development environment and API on the basic of C++. It includes building blocks of C++ library, which can be used to develop GstarCAD application program, extension GstarCAD classes and protocols, create the new command (the as the build-in command) in GstarCAD. GRX is compatible with ARX which is developed on AutoCAD® on the Source code level. Developers only need few configurations to migrate ARX application for AutoCAD® to GRX application for GstarCAD.

GRX programming environment provides an object-oriented C++ programming interface. Developers can utilize this interface to use, optimize and extend GstarCAD. GRX library contains variety of tools which can be taken to GstarCAD open structure, and can directly access GstarCAD database, graphics system and user-defined commands. In addition, the library can be also used with visual LISP and other programming interface. Developers can accomplish the following work with GRX:

- Access GstarCAD database
- Interact with GstarCAD editor
- Create user interface with MFC
- Support multiple document environment
- Create user defined class.
- Built complex applications.
- Interact with other programming environment

2. Development Environment

- Microsoft Visual Studio Enterprise 2017（Version 15.9.17）
- CPU：

Basic Requirement：1.6 GHz CUP，Recommend：3.0 GHz CPU and above

- RAM：

Basic Requirement：2 GB，Recommend：8 GB and above

- Operation System

Windows 10 version 1507 and advanced version：Including home version, professional version, education version and enterprise version（not support LTSC and Windows 10 S）

Windows 8.1（with updated 2919355）： Core version, professional version and enterprise version

Windows 7 SP1（with latest Windows updated）： Including home version, professional version, enterprise version and Ultimate version

- Monitor Resolution:

Traditional monitor：1028 x 800 and above CRT monitor, High resolution and 4K (3840 x 2160) monitor

- GRX SDK 2020
- GstarCAD 2020
- .NET Framework 4.8 and above

3. Install GRX SDK

Download GstarCAD SDK package from our official website:

https://www.gstarcad.net/download/

After you get the GRXSDK.ZIP file, please extract it to your disk directory, for instance, C:\grxsdk. Afterwards you will get 6 files, which they are inc,inc-x64,inc-x86,lib-x64,lib-x86 and samples.

- The Inc file includes header files
- The inc-x64 file includes the files used in COM and .NET for 64 bit
- The inc-x86 file includes the files used in COM and .NET for 32 bit
- The lib-x64 file includes the files referenced from GRX library
- The lib-x86 file includes the files referenced from GRX library
- Samples includes project files, they are dotnet，fact_dg，HelloADS，HelloARX，SimplePalette, etc.

The instruction of sample projects:

- Dotnet includes multiple dotnet samples，including addline，hello，vbhello, etc.
  1) Addline includes .NET example for adding solid line.
  2) Hello includes .NET example for prompting the information of output.
  3) Vbhello includes VB.NET example for how to develop with .NET API.
- fact_dg includes example for defining LSP function.
- HelloADS includes the project for how to develop the type of ADS.
- HelloARX includes includes the example for how to develop with ARX API.
- SimplePalette includes the example for how to develop the type modules for palette.

4. Development Instruction

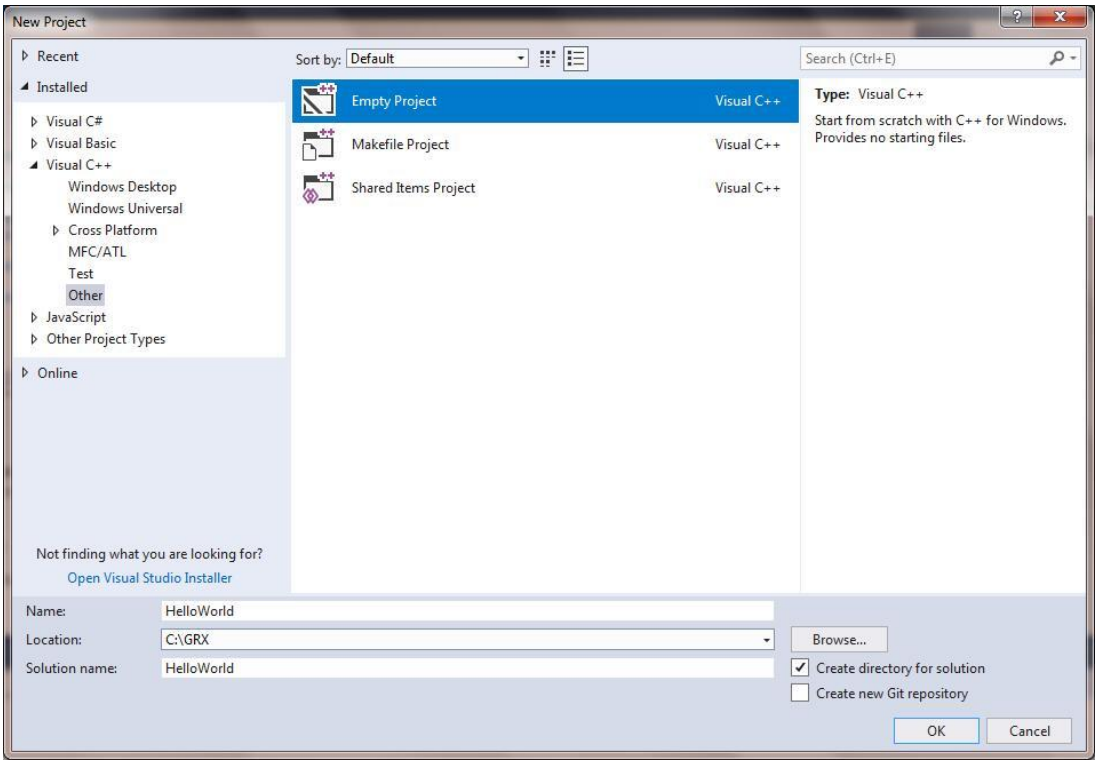
4.1. Create a GRX Project under win 64-Bit

4.1.1. Run Microsoft Visual Studio 2017

Please click [File] → [New] → [Project]. After clicked the menu Project, the New Project dialog box will pops out. Then select Visual C++ in the catalogue Installed Templates and click [Empty Project].

4.1.2. Input Project Saved Path and Project Name

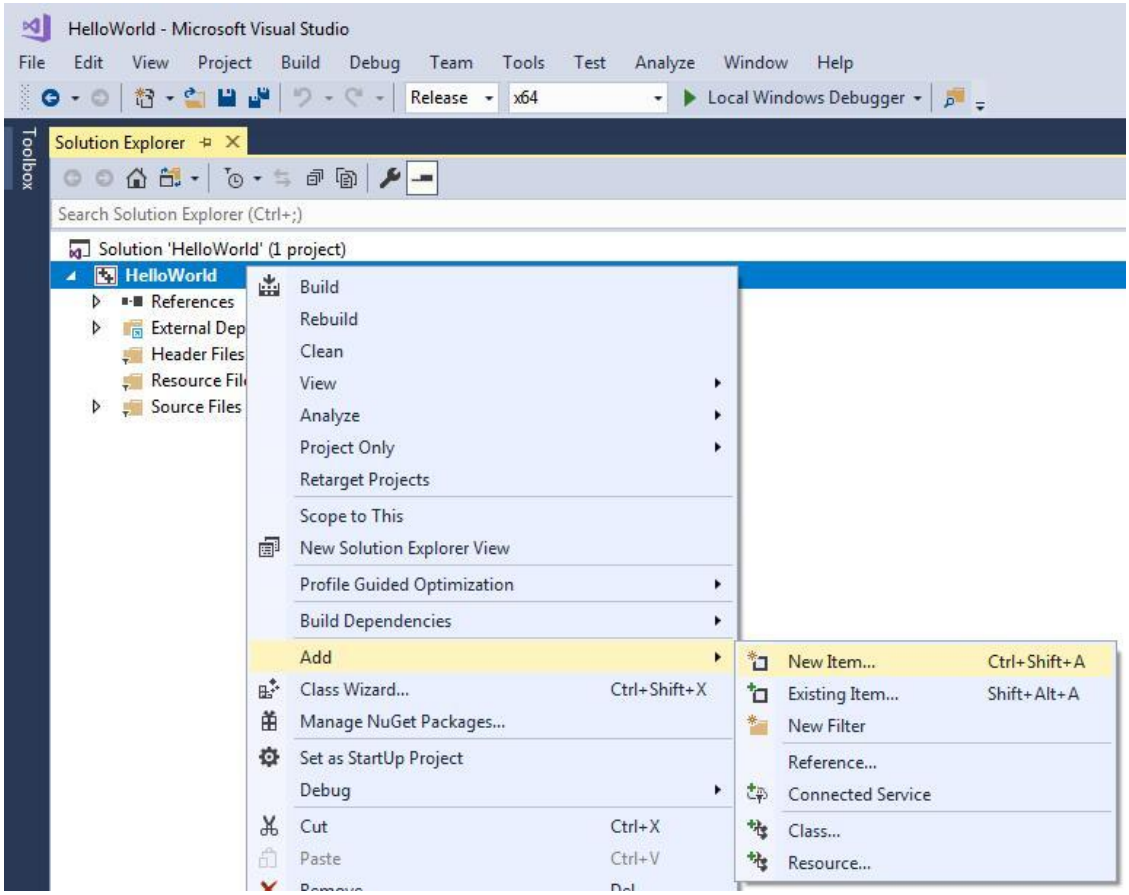
Input "HelloWorld" at the name label in "New Project" dialog box and click OK button to create a new project.

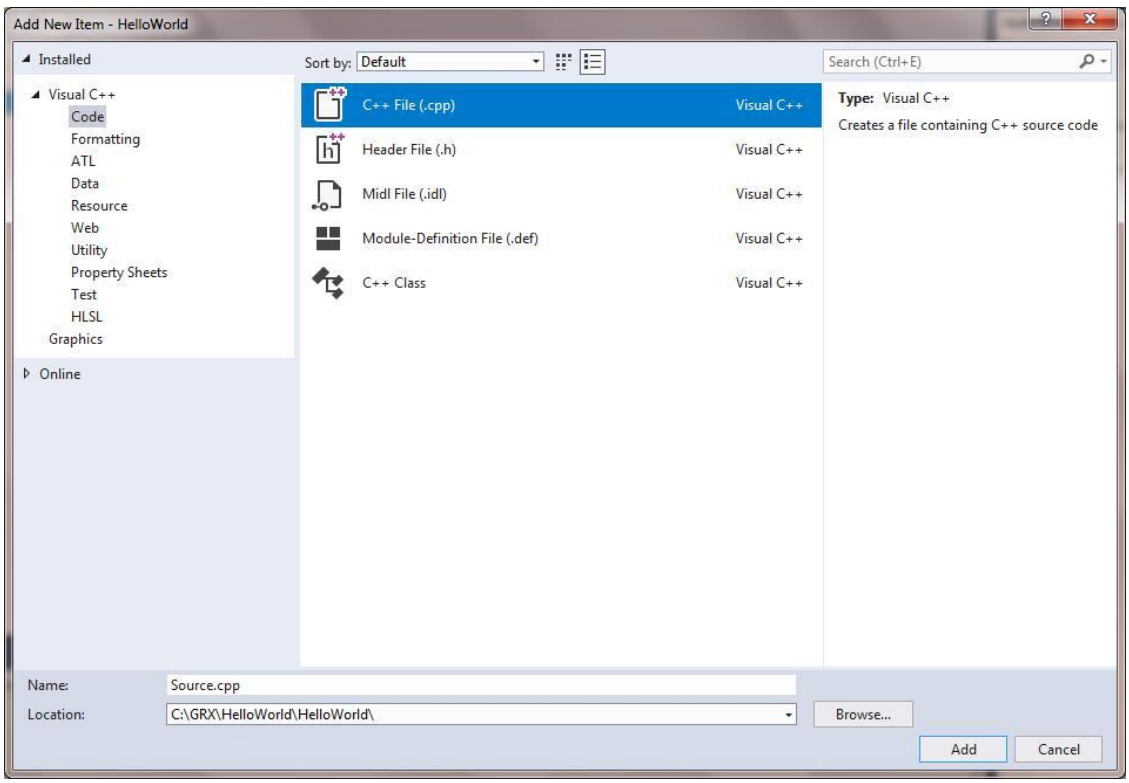


4.1.3. Create a New cpp File

In VS 2017, find the HelloWorld project you have been created in the solution explorer. Select the HelloWorld project and right click on it, then select Add -> New Item. After Add

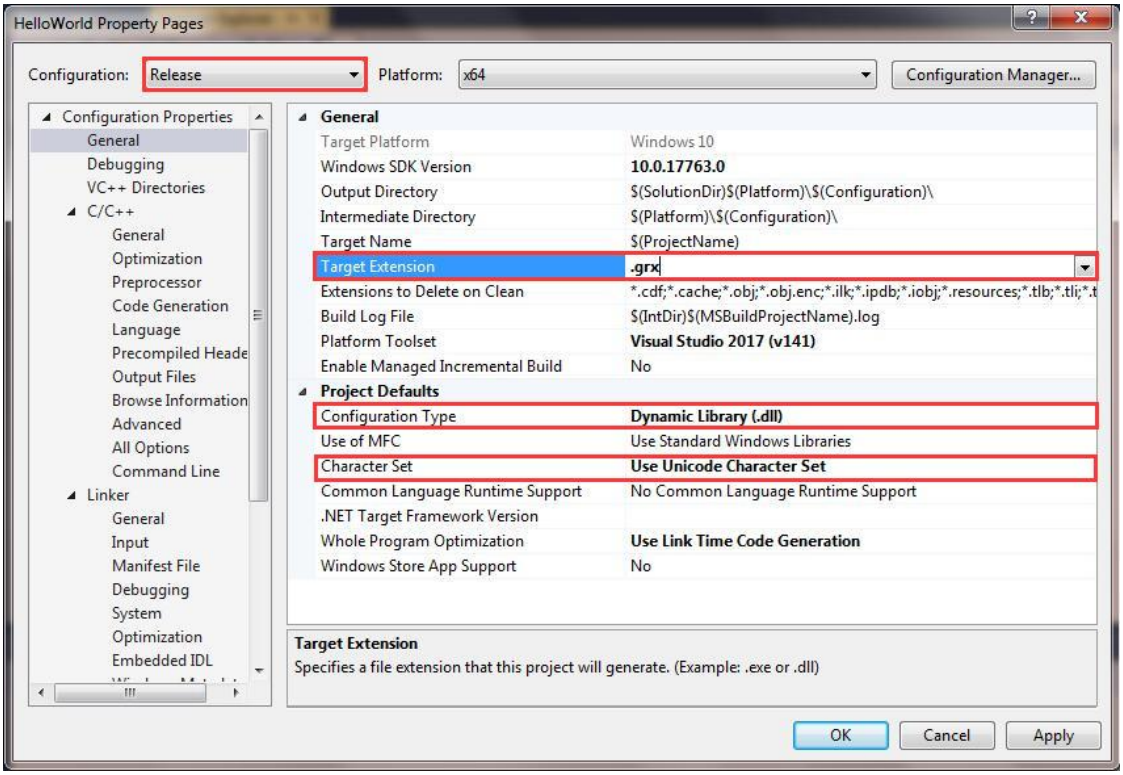
New Item – HelloWord dialog box popped out, select the Code node under Visual C++, then select the C++ File(.cpp) in the middle.





4.1.4. Set the Compile and Link Configuration

Select the project HelloWorld in solution explorer select Property at the right click menu. After selected the Properties option, the HelloWorld Properties Pages will pops out.



1) Select General node under the Configuration Properties and set the configurations as below:

   Set Target Extension as: .grx

   Set Configuration Type as: Dynamic Library (.dll)

   Set Character Set as: Use Unicode Character Set

2) Select General node under the C/C++ node and set the configurations as below:

   Set General-Additional Include Directories as: C:\grxsdk\inc\arx

   Add a configuration at Preprocessor-Preprocessor Definitions:

   _TOOLKIT_IN_DLL_

   Please note that if your Configuration is Debug, you have to delete the _DEBUG from the list. Then select Code Generation node under C/C++, and change Runtime Library to Multi-threaded Debug DLL (/MD).

Set Language-Conformance mode as: No

3) Select Linker and set the configurations as below:

Set General-Additional Library Directories as (If 32-Bit): C:\grxsdk\lib-x86

Set General-Additional Library Directories as (If 64-Bit):C:\grxsdk\lib-x64

Set Input-Additional Dependencies as：

grxport.lib;Td_Root.lib;Td_DbRoot.lib;Td_Db.lib;Td_Ge.lib;Td_Gi.lib;Td_Gs.lib;gcad.lib;gcap.lib;gcdb.lib;gced.lib;gcgs.lib;gcut.lib;gcui.lib

Remark：grxport.lib should be added at the first place.

Set Input-Module Definition File as：C:\grxsdk\inc\arx\RxExport.def

4) Click "OK" button to finish the compiler and linker configuration.

5) Compile your project and make sure it running without error, otherwise, please repeats the steps above.

4.1.5. Add Codes

Please add the following code to HelloWorld.cpp file.

```
#include "windows.h"
#include <tchar.h>
#include <arxHeaders.h>

void initApp();
void unloadApp();
void HelloWorld();

void initApp()
{
    //register a command with the GstarCAD command mechanism
    acedRegCmds->addCommand(_T("HELLOWORLD_CMDS"), _T("Hello"), _T("Hello"), ACRX_CMD_TRANSPARENT, HelloWorld);
}
```

```
    void unloadApp()
    {
        acedRegCmds->removeGroup(L"HELLOWORLD_CMDS");
    }
    void HelloWorld()
    {
        //print "Hello World" in GstarCAD command line
        acutPrintf(_T("\nHello World!"));
    }

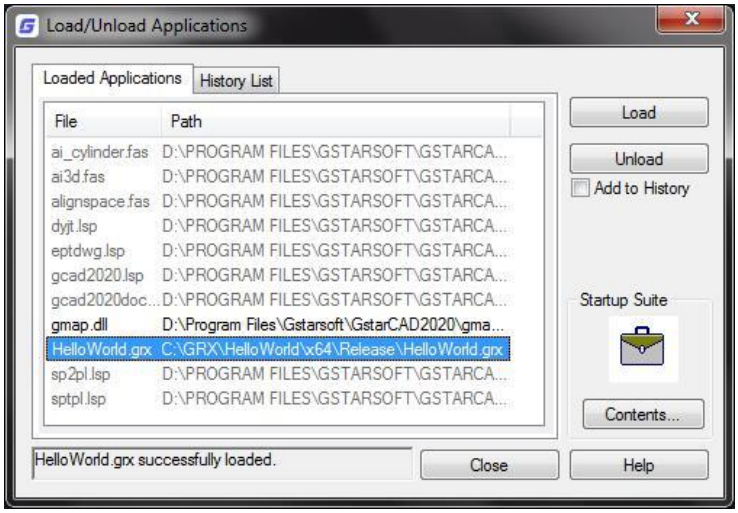
    extern "C" AcRx::AppRetCode gcrxEntryPoint(AcRx::AppMsgCode msg, void
*pkt)
    {
        switch (msg)
        {
        case AcRx::kInitAppMsg:
            acrxDynamicLinker->unlockApplication(pkt);
            acrxDynamicLinker->registerAppMDIAware(pkt);
            initApp();
            break;
        case AcRx::kUnloadAppMsg:
            unloadApp();
            break;
        default:
            break;
        }
        return AcRx::kRetOK;
    }
```

4.1.6. Compile Programming

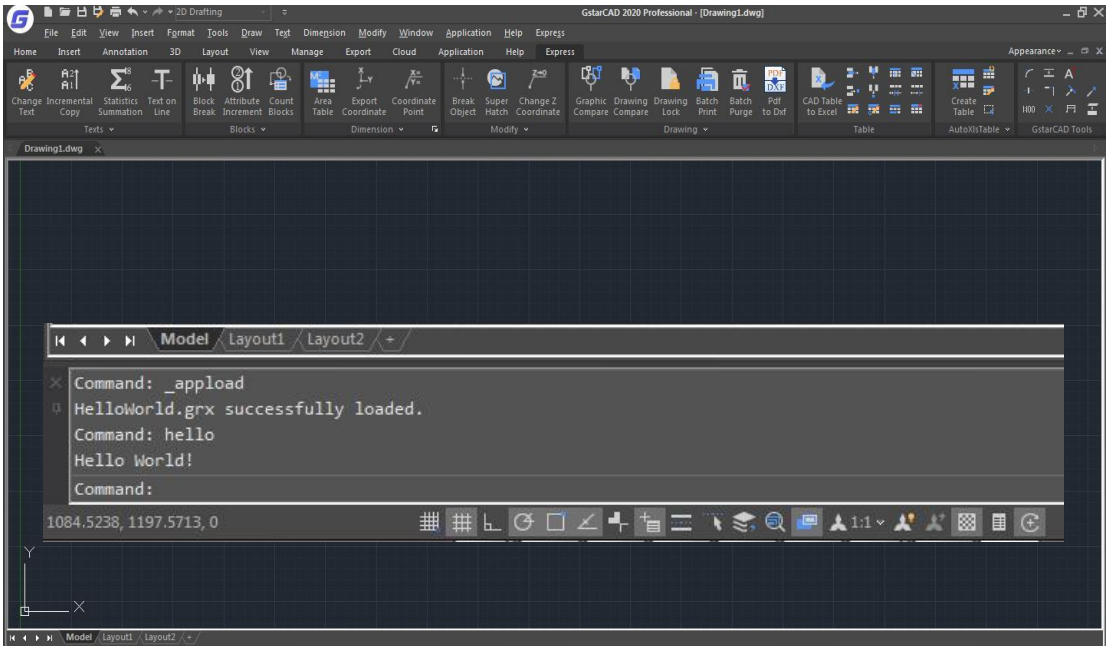
Click Visual Studio 2017 menu item [Build] » [Build solution], then you will get a “HelloWorld.grx” file from C:\GRX\HelloWorld\Release directory.

4.1.7. Run Program

Run GstarCAD and input “appload” at the command line, or select the menu item [Tools] » [loading applications], there will be a [load application files] dialog box. Click Load and find the generated file “HelloWorld.grx” to load it.



Input "hello" at command line, the “Hello World!” will output at command line.



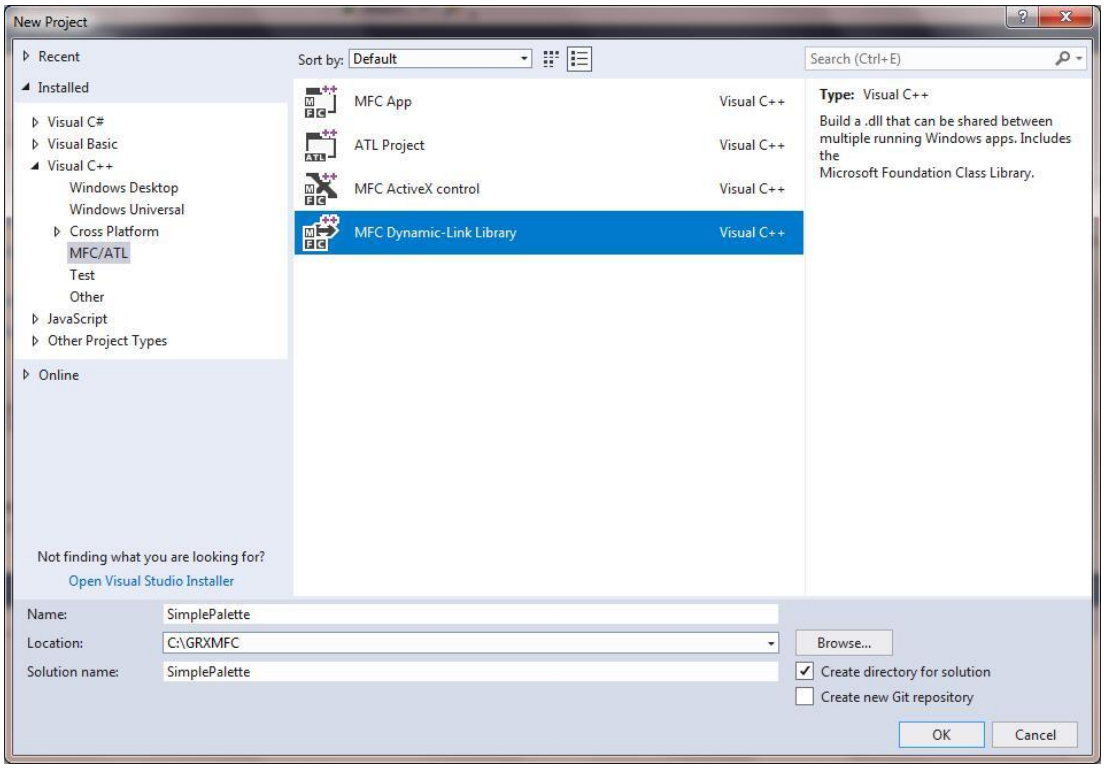
4.2. GRX Program with MFC Library

4.2.1. Run Microsoft Visual Studio 2017

Please click [File] → [New] → [Project]. After clicked the menu Project, the New Project dialog box will pops out. Then select Visual C++ in the catalogue Installed Templates and choose MFC/ATL in the pull down list. Then click MFC Dynamic-Link Library (DLL).

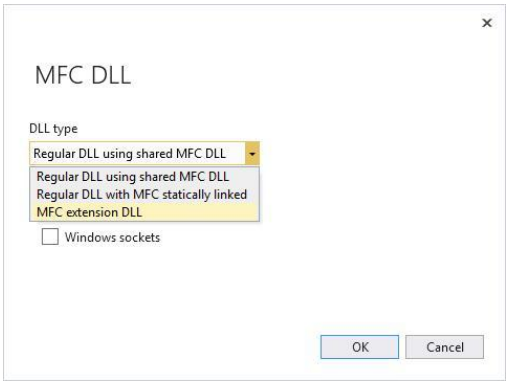
4.2.2. Input Project Saved Path and Project Name

Input "SimplePalette" at the name label in "New Project" dialog box, as shown below.



4.2.3. Choose DLL Type

Click OK button, the "MFC DLL" dialog box will pups out. Please select "MFC Extension DLL" and click OK button.

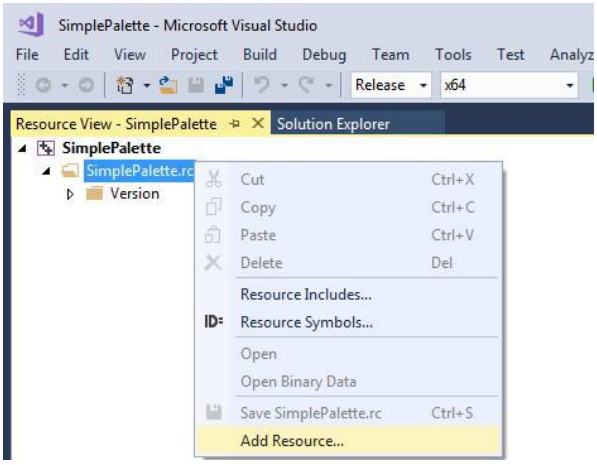


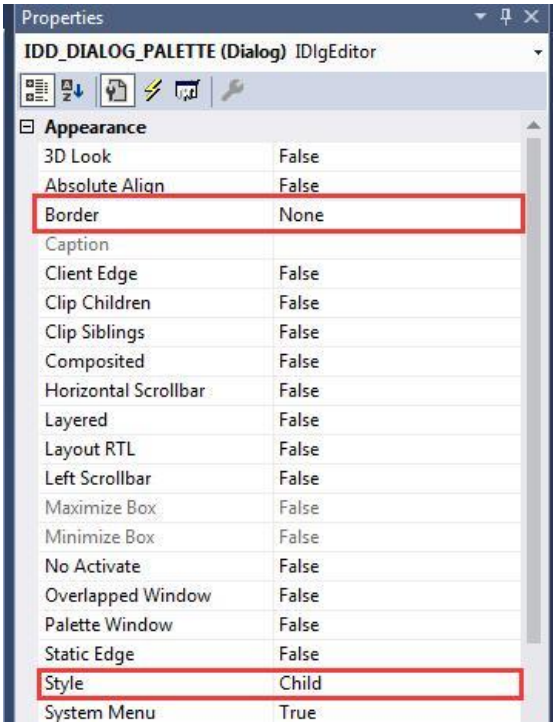
4.2.4. Finish the Project Creation

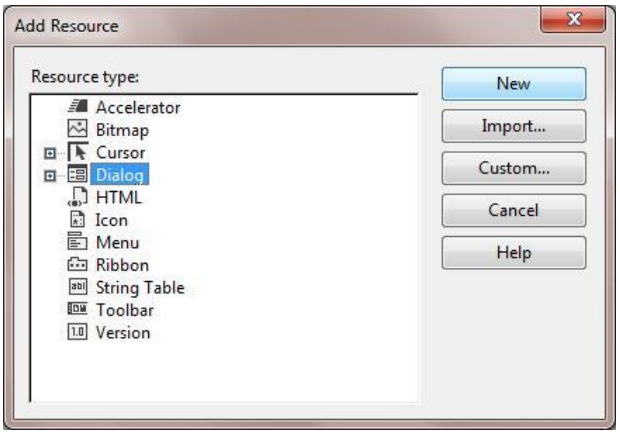
Click OK button, which means a new project has been created.

4.2.5. Create a New Resource and Corresponding Class

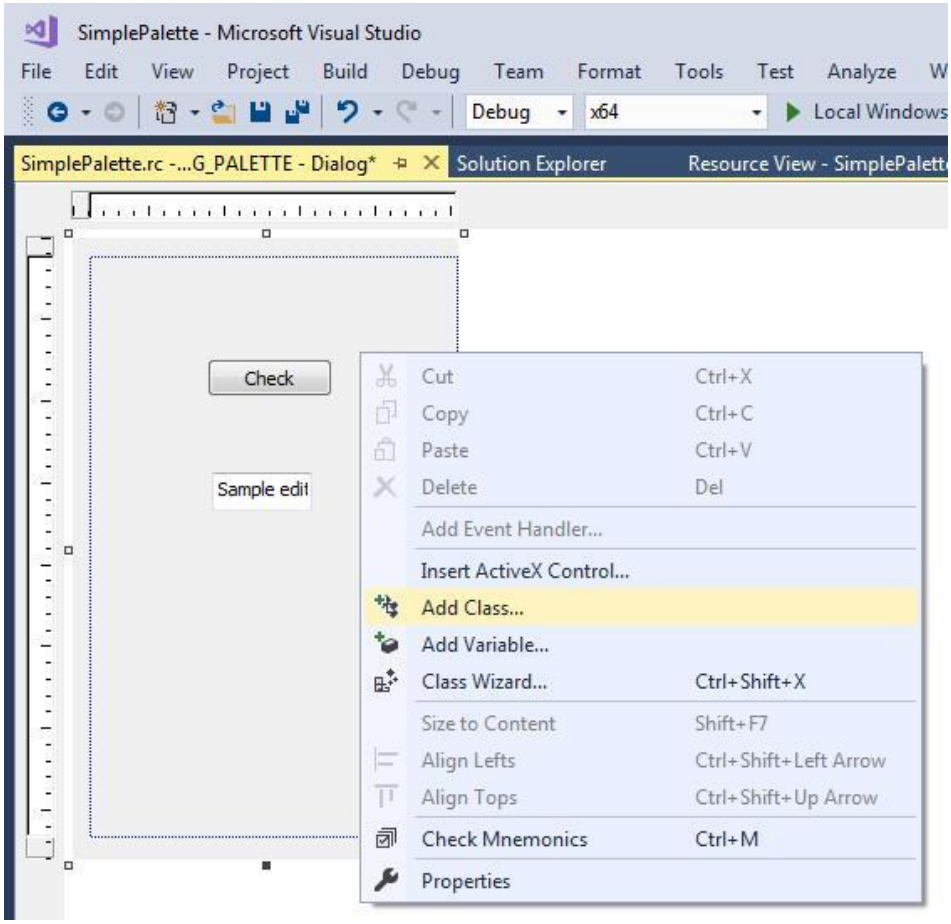
In VS 2017, find the Resource View and select SimplePalette.rc, click Add Resource on right click menu. Select dialog and click New at Add Resource dialog box as shown bleow. Select IDD_DIALOG1 on Resource View and right click on it, then select Properties to change ID as IDD_DIALOG_PALETTE. Right click the dialog you just created, and select Properties. Change Border as None and Style as Child. Afterwards edit control and button from Toolbox (Ctrl+Alt+X). Change the properties of button caption as Check and ID as IDC_BUTTON_CHECK.

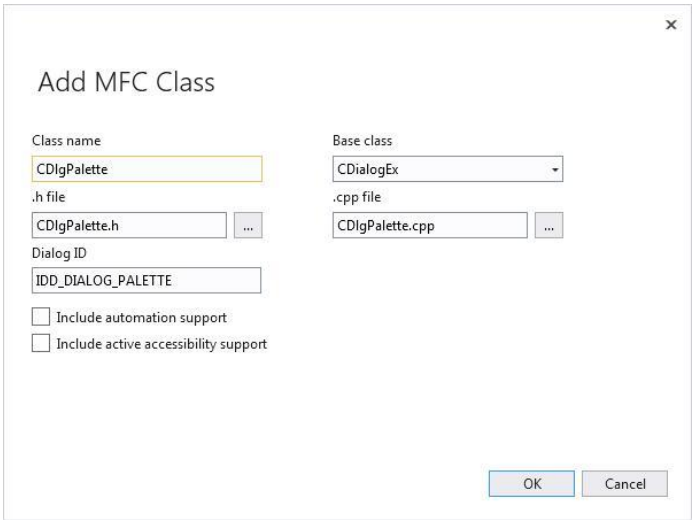






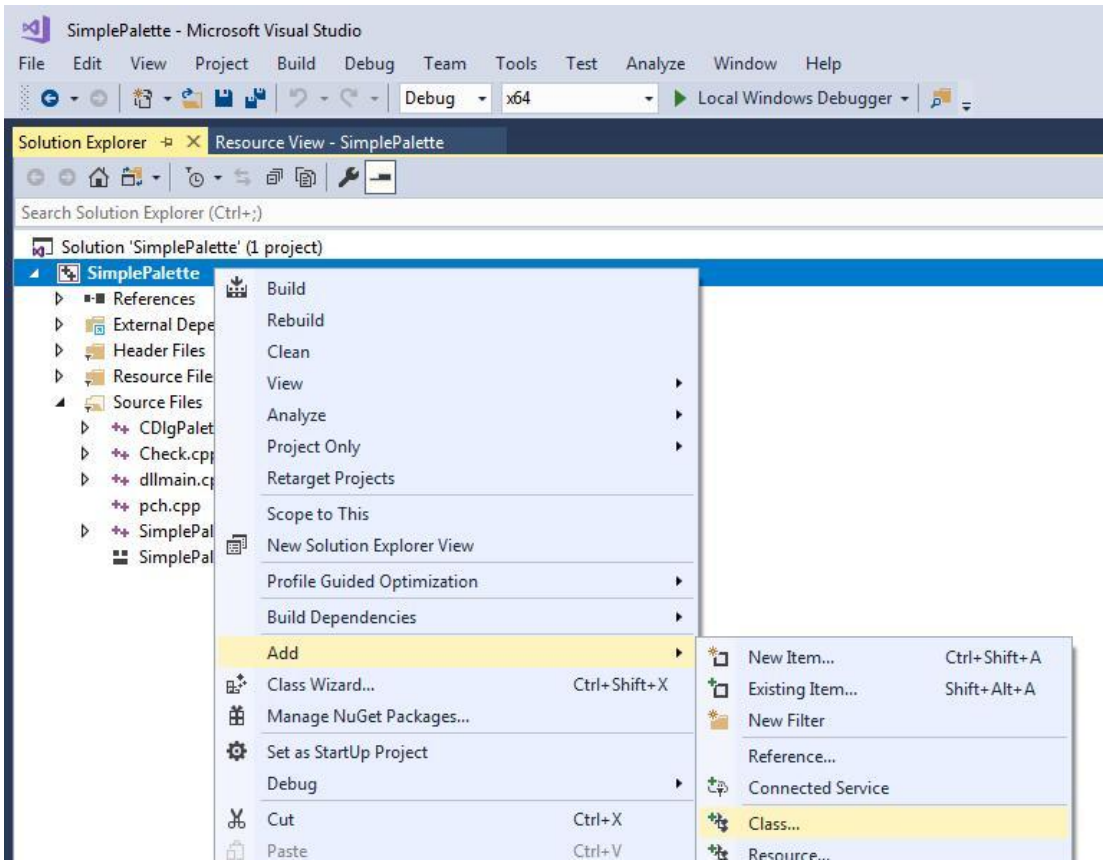
Select the dialog and right click on it, then select the Add Class. Set Class Name as CDlgPalette and click the OK button.

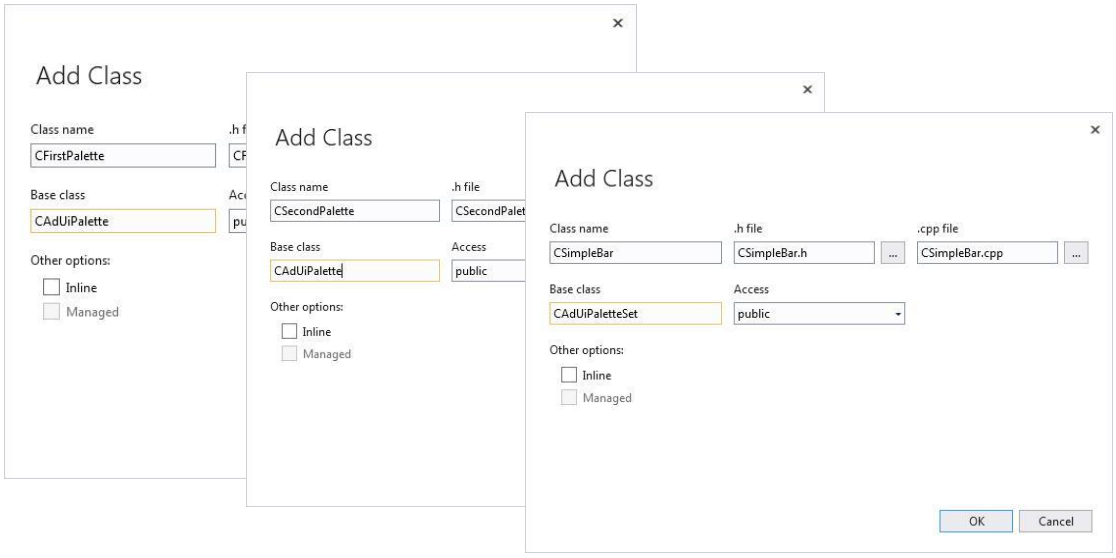




4.2.6. Create Three New C++ Class Files

Select the project SimplePalette in solution explorer of VS2017 and right click to add class. Set the name of the class as CFirstPalette, CSecondPalette and CSimpleBar respectively, and set the Base class as CAdUiPalette, CAdUiPalette and CAdUiPalette respectively.





4.2.7. Add Codes

Add code below into the bottom of the file stadfx.h, you can find it at Head File in Solution Explore.

```
#include <arxHeaders.h>
```

Class code for CDlgPalette, CFirstPalette, CSecondPalette and CSimpleBar could refer to DlgPalette, FirstPalette, SecondPalette and SimpleBar files from example program (C:\grxsdk\samples\SimplePalette).

Add the following code to SimplePalette.cpp file:

```
#include "stdafx.h"
#include "SimpleBar.h"

CSimpleBar* gpBar = NULL;

static void doIt()
{
  if (gpBar)
  {
    gpBar->ShowPane(TRUE, FALSE, TRUE);
  }
  else
  {
    acedInitGet(0, L"Float Dock");
    ACHAR result[132] = {0};
    int rc = acedGetKword(L"\nFloat or Docking?", result);
    bool bFloat = true;
    if (rc == RTNONE)
```

```
        {
            bFloat = true;
        }
        else if (rc != RTNORM)
        {
            return;
        }
        else
        {
            bFloat = _wcsicmp(result, L"Float") == 0;
        }

        CSize sizeDefault(200, 450);
        CRect rcDefault(CPoint(150,200), sizeDefault);

        gpBar = new CSimpleBar;
        gpBar->SetMinSize(sizeDefault);

    DWORD dwStyle = WS_CHILD | WS_VISIBLE | CBRS_SIZE_DYNAMIC |
                                CBRS_GRIPPER;
        if (bFloat)
        {
            dwStyle |= CBRS_FLOATING;
        }
        else
        {
            dwStyle |= CBRS_RIGHT;
        }
```

```
            gpBar->Create(L"Simple", dwStyle,
               rcDefault, acedGetAcadFrame());
      gpBar->EnableDocking(CBRS_ALIGN_LEFT    | CBRS_ALIGN_RIGHT |
                                        CBRS_ALIGN_BOTTOM);
            if (bFloat)
            {
               gpBar->FloatPane(rcDefault, DM_SHOW, true);
            }
            else
            {
               acedGetAcadFrame()->DockPane(gpBar);
            }
         }
      }

      void initApp()
      {
         acedRegCmds->addCommand(_T("ASDK_SAMPLES_SIMPLEPALETTE"),
            _T("ASDK_SIMPLEPALETTE"), _T("SIMPLEPALETTE"),
ACRX_CMD_MODAL,
            doIt);
      }



      void unloadApp()
      {
         acedRegCmds->removeGroup(_T("ASDK_SAMPLES_SIMPLEPALETTE"));

         if (gpBar)
```

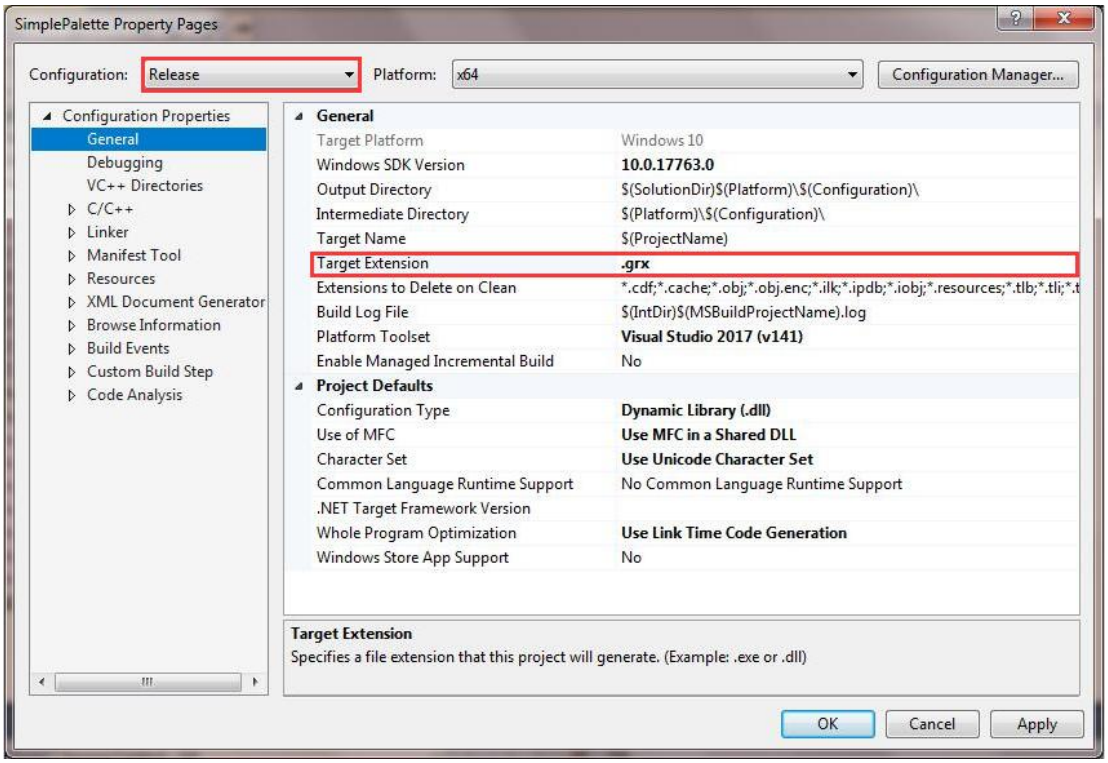
```
    {
        gpBar->DestroyWindow();
        delete gpBar;
        gpBar = NULL;
    }
}



extern "C" AcRx::AppRetCode
    acrxEntryPoint(AcRx::AppMsgCode msg, void* appId)
{
    switch(msg) {
    case AcRx::kInitAppMsg:
        acrxDynamicLinker->unlockApplication(appId);
        acrxDynamicLinker->registerAppMDIAware(appId);
        initApp();
        break;
    case AcRx::kUnloadAppMsg:
        unloadApp();
    }
    return AcRx::kRetOK;
}
```

4.2.8. Set Compile and Link Item

1) Select the project SimplePalette in solution explorer select Property at the right click menu. After selected the Properties option, the SimplePalette Properties Pages will pops out.



2) Select General node under the Configuration Properties and set the configurations as below:

Target Name set to：.grx

Configuration Type set to：Dynamic Library (.dll)

Character Set set to: Use Unicode Character Set

3) Select General node under the C/C++ node and set the configurations as below:

Set General-Additional Include Directories as: C:\grxsdk\inc\arx

Add a configuration at Preprocessor-Preprocessor Definitions:

_TOOLKIT_IN_DLL_

Please note that if your Configuration is Debug, you have to delete the _DEBUG from the list. Then select Code Generation node under C/C++, and change Runtime Library to Multi-threaded Debug DLL (/MD).

Set Language-Conformance mode as: No

4) Select Linker and set the configurations as below:

Set General-Additional Library Directories as (If 32-Bit): C:\grxsdk\lib-x86

Set General-Additional Library Directories as (If 64-Bit):C:\grxsdk\lib-x64

Set Input-Additional Dependencies as：

grxport.lib;Td_Root.lib;Td_DbRoot.lib;Td_Db.lib;Td_Ge.lib;Td_Gi.lib;Td_Gs.lib;gcad.lib;gcap.lib;gcdb.lib;gced.lib;gcgs.lib;gcut.lib;gcui.lib

Remark：grxport.lib should be added at the first place.

Set Input-Module Definition File as：C:\grxsdk\inc\arx\RxExport.def

5) Click "OK" button to finish the compiler and linker configuration.

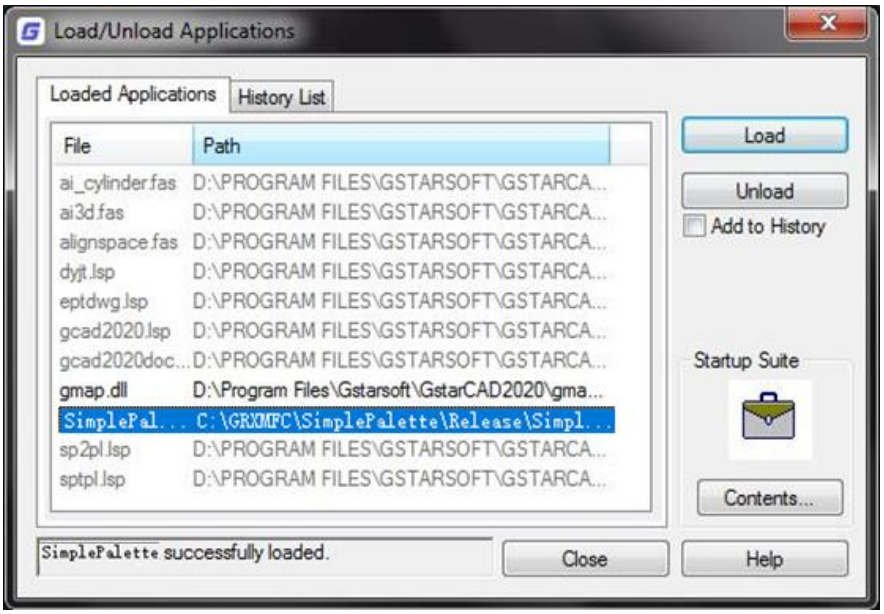
6) Compile your project and make sure it running without error, otherwise, please repeats the steps above.

4.2.9. Compile Programming

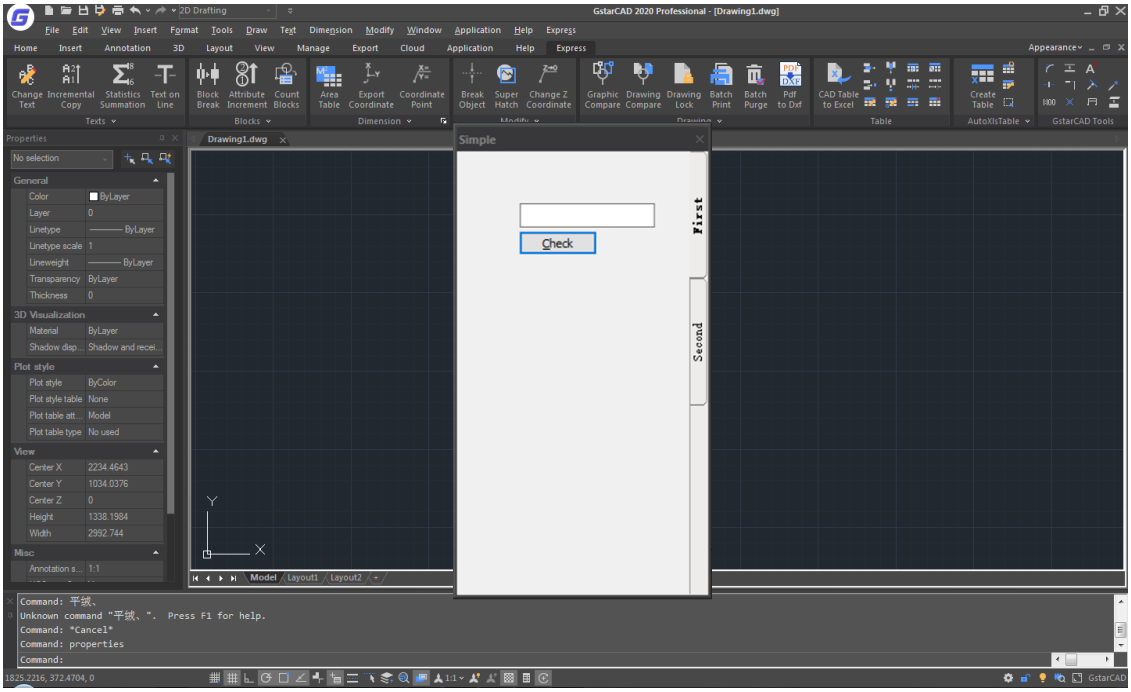
Click Visual Studio 2017 menu item [Build] » [Build solution], then you will get a “SimplePalette.grx” file from C:\GRX\ SimplePalette \Release directory.

4.2.10. Run Program

Run GstarCAD and input “appload” at the command line, or select the menu item [Tools] » [loading applications], there will be a [load application files] dialog box. Click Load and find the generated file “SimplePalette.grx” to load it.



Input "simplepalette" in command line, the “Float or Dock?” will output at command line. Press Enter on keyboard and the palette will be shown at GstarCAD workspace.

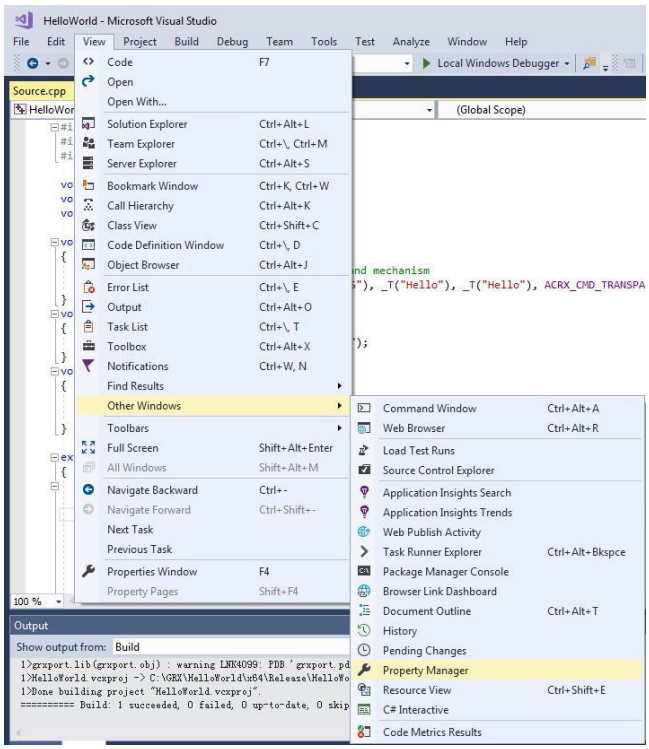

GstarCAD 2020 Professional - [Drawing1.dwg]
Simple
Check
First
Second
Drawing1.dwg
Command: 平铖、
Unknown command "平铖、". Press F1 for help.
Command: *Cancel*
Command: properties
Command:

5. What is Project Property Sheet File

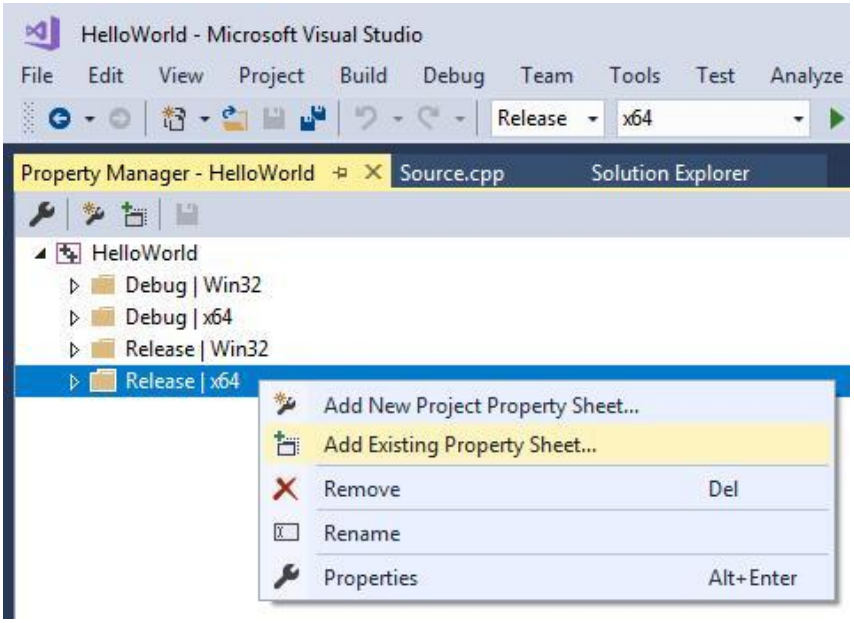
Project Property Sheet File is used to save the project classes which can be reused and shared. It can avoid to modify project property by mistake. The project properties can be independently applied to generate the configuration (debug or release) and target platform (Win32, x64 or ARM) in any combination.

6. How to Use Project Property Sheet File

1) Use Project Property sheet to reset the configuration, we can take" HelloWorld" project as a sample. Open "HelloWorld" project, click [View] » [Other Windows] » [Properties Manager], as shown below.



2) In "Properties Manager "dialog box, right click on Release| x64, and then click Add Existing Property Sheet, as shown below.



3) Add C:\grxsdk\inc\arx\arx.props property sheet file (this file contains dbx.props property sheet file, and add Td_Root.lib;Td_DbRoot.lib;Td_Db.lib;Td_Ge.lib;Td_Gi.lib;Td_Gs.lib to Additional Dependencies). Add C:\grxsdk\inc\arx\rxsdk_releasecfg.props with the same steps(this file contains rxsdk_common.props property sheet file and add "Additional Include Directories and Additional Libray Directories"). In addition, C:\grxsdk\inc\arx\rxsdk_debugcfg.props is necessary if under debug project.
4) In Visual Studio 2017 "Solution Explore", right click on the “HelloWorld” project, and click Properties at context menu. Click C/C++,and then click General, change the Additional Include Directories to C:\grxsdk\inc\arx. Click Linker and click Input, change Additional Dependencies to grxport.lib, change Module Definition File to C:\grxsdk\inc\arx\RxExport.def
5) Then Compile.

7. GstarCAD Programming General Method to Adapt 4K Monitor

7.1. Development Background and Target

GstaCAD provides implementation method to support 4K monitor.

7.2. Implementation Method

7.2.1. Dialog Box, User-Defined Window, Control Adaptive

- For all the User-defined window, dialog box and control in the program, it is no needs to adapt if they are from resource manage. Only the ones which have been modified through module need to be adapt. For example, If the button has been stretched by 5 units, then the “5” here should be adapted. You can use the interface " int GdUiGetCtrlDPISize(const int val)" to adapt this kind of module. Input the module value and the returned value is the adapted result. Replace the original module with GdUiGetCtrlDPISize (module). The interface is in gcui, and the project needs to rely on gcui.lib when using it.
- For custom Combox and Listbox, if they do not have 4k adaptation effect, then the MeasureItem interface needs to be rewritten. In this way, the heights of these controls can be adjusted to 4k effect, and the original height also can be adjusted to 4k effect in the logic.

7.2.2. Image Adaptation

To adapt the bitmap image, you need to use the interface “void GdUiScaleBitmapDPI (CBitmap * pBitmap, CBitmap & bitmapNew)”. The export parameter bitmapNew will return the images after adaptation. If the UI with bitmap has not been adapted for 4 k monitor, then this interface can be helpful.

8. The differences between GRX and ARX interface

8.1. Function: monitorInputPoint

For monitorInputPoint function, the statement between GRX and ARX is different. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
#ifdef GCAD_CORE
Acad::ErrorStatus AsdkInputPointMonitor::monitorInputPoint(
        bool& appendToTooltipStr,
        ACHAR*& additionalTooltipString,
        AcGiViewportDraw* drawContext,
        AcApDocument* document,
        bool pointComputed,
        int history,
        const AcGePoint3d& lastPoint,
        const AcGePoint3d& rawPoint,
        const AcGePoint3d& grippedPoint,
        const AcGePoint3d& cartesianSnappedPoint,
        const AcGePoint3d& osnappedPoint,
        AcDb::OsnapMask osnapMask,
        const AcArray<AcDbCustomOsnapMode*>& customOsnapModes,
        AcDb::OsnapMask osnapOverrides,
        const AcArray<AcDbCustomOsnapMode*>& customOsnapOverrides,
        const AcArray<AcDbObjectId>& apertureEntities,
        const AcArray<AcDbObjectIdArray,
OdObjectsAllocator<AcDbObjectIdArray > >& nestedApertureEntities,
        const AcArray<Hccad::GsMarker>& gsSelectionMark,
        const AcArray<AcDbObjectId>& keyPointEntities,
        const AcArray<AcDbObjectIdArray,
```

```
OdObjectsAllocator<AcDbObjectIdArray > >& nestedKeyPointEntities,
                const AcArray<Hccad::GsMarker>& keyPointGsSelectionMark,
                const AcArray<AcGeCurve3d*>& alignmentPaths,
                const AcGePoint3d& computedPoint,
                const ACHAR* tooltipString)
        #else
        virtual Acad::ErrorStatus monitorInputPoint(
                bool& appendToTooltipStr,
                TCHAR*& additionalTooltipString,
                AcGiViewportDraw* drawContext,
                AcApDocument* document,
                bool pointComputed,
                int history,
                const AcGePoint3d& lastPoint,
                const AcGePoint3d& rawPoint,
                const AcGePoint3d& grippedPoint,
                const AcGePoint3d& cartesianSnappedPoint,
                const AcGePoint3d& osnappedPoint,
                AcDb::OsnapMask osnapMasks,
                const AcArray<AcDbCustomOsnapMode*>& customOsnapModes,
                AcDb::OsnapMask osnapOverrides,
                const AcArray<AcDbCustomOsnapMode*>& customOsnapOverrides,
                const AcArray<AcDbObjectId>& apertureEntities,
                const AcArray< AcDbObjectIdArray,
                AcArrayObjectCopyReallocator< AcDbObjectIdArray > >&
nestedPickedEntities,
                const AcArray<INT_PTR>& gsSelectionMark,
                const AcArray<AcDbObjectId>& keyPointEntities,
                const AcArray< AcDbObjectIdArray,
                AcArrayObjectCopyReallocator< AcDbObjectIdArray > >&
```

```
nestedKeyPointEntities,
                const AcArray<INT_PTR>& keyPointGsSelectionMark,
                const AcArray<AcGeCurve3d*>& alignmentPaths,
                const AcGePoint3d& computedPoint,
                const TCHAR* tooltipString );
    #endif
```

8.2. Function: intersectWith

For intersectWith function, the return vale of intersectWith is different between GRX and ARX when there is no intersection. GRX also return as “eOK” even there is no intersection. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
bool bHasInters = false;
AcGePoint3dArray aryIntters;
Acad::ErrorStatus es = pCur->intersectWith(pBreak , AcDb::kOnBothOperands , aryIntters);
#ifndef GCAD_CORE
    if ( es != Acad::eOk ) {
    return bHasInters;
    }
#endif
    bHasInters = aryIntters.length() > 0 ;
```

8.3. Function: AcDbField::evaluate

For AcDbField::evaluate function, GRX has to call for the value of the subfield first. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown

below.

Example：

```
AcDbField* pFld = NULL;
if (Acad::eOk == pEnt->getField(_T("TEXT"),pFld,AcDb::kForWrite))
{
#ifdef GCAD_CORE
    // GCAD must call children's evaluate
    int nCount = pFld->childCount();
    for (int i = 0;i<nCount;i++)
    {
        AcDbField* pChiled = NULL;
        if (pFld->getChild(i, pChiled, AcDb::kForWrite) Acad::eOk)
        {
            pChiled->evaluate(AcDbField::kDemand,0);
            pChiled->close();
        }
    }
    pFld->close();
    if (Acad::eOk pEnt->getField(_T("TEXT"),pFld,AcDb::kForWrite))
    {
        if (Acad::eOk == pFld->evaluate(AcDbField::kDemand,0))
        {
            LPTSTR lpstr = NULL ;
            if (Acad::eOk == pFld->getValue(lpstr))
            {
                acutPrintf(L"%s\n", lpstr);
                acutDelString ( lpstr ) ;
                delete lpstr ;
                lpstr = NULL ;
            }
```

```
            }
        }
#else
        if (Acad::eOk == pFld->evaluate(AcDbField::kDemand,0))
        {
            LPTSTR lpstr = NULL ;
            if (Acad::eOk == pFld->getValue( lpstr))
            {
                acutPrintf(L"%s\n", lpstr);
                acutDelString ( lpstr ) ;
                delete lpstr ;
                lpstr = NULL ;
            }
        }
#endif
        pFld->close();
}
```

8.4. Interface for “Plot”

When load Plot interface, the begin Document function in GRX will call beginPage function automatically. Meanwhile, the beginPage will call beginGenerateGraphics function automatically. Which means when you need to load Plot interface, you don’t need to call beginPage, beginGenerateGraphics, endGenerateGraphics and endPage functions any more. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
    AcPlPlotEngine* mPlotEngine;
    es = AcPlPlotFactory::createPublishEngine(mPlotEngine);
```

```
    es = mPlotEngine->beginPlot(NULL);
    AcPlPlotInfo plotInfo;
    AcPlPlotInfoValidator plotInfoValidator;
    plotInfo.setLayout(mPlotSettings->id());
    plotInfo.setOverrideSettings(mPlotSettings);
    plotInfoValidator.setMediaMatchingPolicy(AcPlPlotInfoValidator::kMatchEnabled);
    es = plotInfoValidator.validate(plotInfo);
    es = mPlotEngine->beginDocument(plotInfo, curDoc()->fileName(), NULL,1, true,
_T("c:\\test.eps"));
#ifndef GCAD_CORE
    AcPlPlotPageInfo pageInfo;
    es = mPlotEngine->beginPage(pageInfo, plotInfo, true);
    es = mPlotEngine->beginGenerateGraphics();
    es = mPlotEngine->endGenerateGraphics();
    es = mPlotEngine->endPage();
#endif
    es = mPlotEngine->endDocument();
    es = mPlotEngine->endPlot();
    mPlotEngine->destroy();
```

8.5. Gs initialization

The difference for Gs initialization, please refer to the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
    Acad::ErrorStatus initDrawingCtrl(AcDbDatabase *pDb,const TCHAR *chSpace)
    {
        // have we got a valid drawing
        if (NULL == pDb)
```

```
    {
            return Acad::eNullBlockName;
        }
        // initialize the preview control
        mPreviewCtrl.init();
        mPreviewCtrl.SetFocus();
        AcDbBlockTableRecordPointer bptr(chSpace, pDb, AcDb::kForRead);
        AcDbObjectId idCurrenVs;
        // set the view to the Active AutoCAD view
        idCurrenVs = SetViewTo(mPreviewCtrl.mpView, pDb, m_viewMatrix);
        // tell the view to display this space
        mPreviewCtrl.view()->add(bptr, mPreviewCtrl.model());
        mPreviewCtrl.view()->setVisualStyle(idCurrenVs);
        return Acad::eOk;
    }
#ifdef GCAD_CORE
    AcGePoint3d ptMin ( 1e20, 1e20, 1e20);
    AcGePoint3d ptMax ( -1e20, -1e20, -1e20);
    m_tempExt.set ( ptMin , ptMax ) ;
    OdDbDatabasePtr pDb = acdbHostApplicationServices()->createDatabase();
    initDrawingCtrl(pDb,L"*Model_Space");
#else
    m_tempExt.minPoint().set(1e20, 1e20, 1e20);
    m_tempExt.maxPoint().set(-1e20, -1e20, -1e20);
    AcDbDatabase* pDb = new AcDbDatabase;
    initDrawingCtrl(pDb,L"*Model_Space");
#endif
```

8.6. Gs Pan

When you need to call Gs Pan function, it's different between GRX and ARX. You have to use it upside down. Use the GCAD_CORE macro to distinguish the different uses of GRX and ARX as shown below.

Example：

```
#ifdef GCAD_CORE
    AcGeVector3d pan_vec( -(point.x-mStartPt.x), -(point.y-mStartPt.y), 0 );
#else
    AcGeVector3d pan_vec( -(point.x-mStartPt.x), point.y-mStartPt.y, 0 );
#endif
    pan_vec.transformBy(mpView->viewingMatrix() *
mpView->worldToDeviceMatrix().inverse());
    mpView->dolly(pan_vec);
    Invalidate();
mStartPt = point;
```

8.7. acedGetArgs() release issue

The resbuf returned by acedGetArgs () cannot be released. (No need to call acutRelRb ()).

9. Comparison of GRX and ARX libraries

9.1. GRX class library description

The following libraries are frequently used in GRX. These libraries have the similar effect with the corresponding libraries under ARX, mainly including:：

- GcRx- has the similar effect with the AcRX library, which is used to bind an application and register and identify for the running class.。
- GcEd- has the similar effect with the AcEd library, used to register custom commands and event notifications.
- GcDb- has the similar effect with AcDb library, it's GstarCAD database class.
- GcGi- has the similar effect with the AcGi library and is used for the graphics class of GstarCAD.
- GcGe- has the similar effect with the AcGe library, and is used for general linear algebra calculation and application classes of geometric objects.

9.1.1. GcRx

The GcRx class library is used for DLL initialization and the system level for running class registration and identification. It provides the functions below:

- Class identification and inheritance analysis of object runtime.
- Add a new protocol to an existing class at runtime.
- Object equality and composition judgment.
- Object copy.

9.1.2. GcEd

The GcEd class library is used to define and register new GstarCAD commands, which are the same as GstarCAD internal/original commands.

9.1.3. GcDb

The GcDb class library represents the classes that build up the GstarCAD database. The database stores the information of all graphic objects. These graphic objects are called “Entities”. These graphic objects and non-graphic entities (such as layers, line types, and text styles) constitute GstarCAD graphics.

9.1.4. GcGi

GcGe library provides the graphic interface which is used to draw CAD entities.

9.1.5. GcGe

GcGe library provides some geometric tools (e.g., vector and matrix) to perform the 2D and 2D geometric operations, it also provides the basic geometric objects, such as point, curve and curved surface.

10. Copyright

Copyright reserved: Xi'an Gstarsoft Co.,Ltd

Usage License: Allows copying, quote any part of this document. Changes of any part of this document are forbidden without authorization. Please keep this statement when copying and quoting it, otherwise it will be held liable.

* AutoCAD® is a product of Autodesk® Company which is one of the CAD software solution providers. ARX is an abbreviated name of ObjectARX, which is AutoCAD® Runtime eXtension, and C++ SDK provided by Autodesk®